

Control de tracción y control de velocidad para un auto a control remoto

Resumen

El proyecto consiste de un pequeño auto a control remoto que implementa un sistema de control de tracción (TC) y un sistema de control de velocidad (CC). El sistema de control de tracción logra un deslizamiento controlado de las ruedas en situaciones de baja tracción mediante un control preciso de la potencia de los motores. Por otro lado, el sistema de control de velocidad mantiene una velocidad constante, determinada por el usuario, incluso cuando cambian las condiciones de manejo (por ejemplo, al pasar de un terreno horizontal a uno en pendiente). El auto no tiene dirección, solo se mueve en línea recta (en ambas direcciones). Se maneja con un joystick mediante comandos bluetooth. El joystick se conecta a un software de PC, el cual manda los correspondientes mensajes al auto mediante bluetooth. Este software también recibe telemetría enviada por el auto, y la muestra visualmente de forma similar a la de un osciloscopio. Dos ruedas tienen tracción (directa), las otras dos son libres. La detección de patinaje se hace mediante la utilización de cuatro encoders incrementales ópticos (uno por rueda).

Objetivos

Objetivo general

Implementar un sistema de control de tracción para mejorar la aceleración de un auto a control remoto ante una situación de patinaje. En general, la mejor aceleración posible se obtiene cuando las ruedas experimentan un poco de deslizamiento (entre 5% y 10%). Es decir, se busca implementar un deslizamiento controlado de las ruedas.

También se busca implementar un control de velocidad que permita configurar la velocidad deseada, de forma que la computadora del auto se encargue de manejar la aceleración.

Además de esto, se desea un sistema de telemetría que permita visualizar gráficamente en la PC varios valores de relevancia. Por ejemplo: PWM enviados a cada motor, RPMs medidas, RPM deseada, etc.

Objetivos específicos

- Medir las velocidades de las distintas ruedas utilizando encoders incrementales.
- Detectar patinaje de las ruedas utilizando los datos de los encoders.
- Ajustar la potencia de los motores en base a la aceleración deseada por el usuario (i.e., que tanto intenta acelerar con el joystick), pero limitada por el control de tracción cuando sea necesario para evitar patinaje. La

potencia de los motores se regula mediante PWM. Esta parte requerirá de un controlador PID para lograr el deslizamiento óptimo de las ruedas. Si el control de velocidad se encuentra activado, entonces los motores deberán mantener una velocidad constante, incluso al someterlos a esfuerzos diferentes (como por ejemplo, intentando frenarlos con la mano). En tal caso, se utilizará el mismo controlador PID.

- Implementar un protocolo de comunicación bluetooth para los comandos de control del auto y para la telemetría. Esto último permitirá observar mediante gráficos los datos que el auto procesa.
- Graficar la telemetría en la PC.

Descripción funcional del sistema

Diagrama de bloques

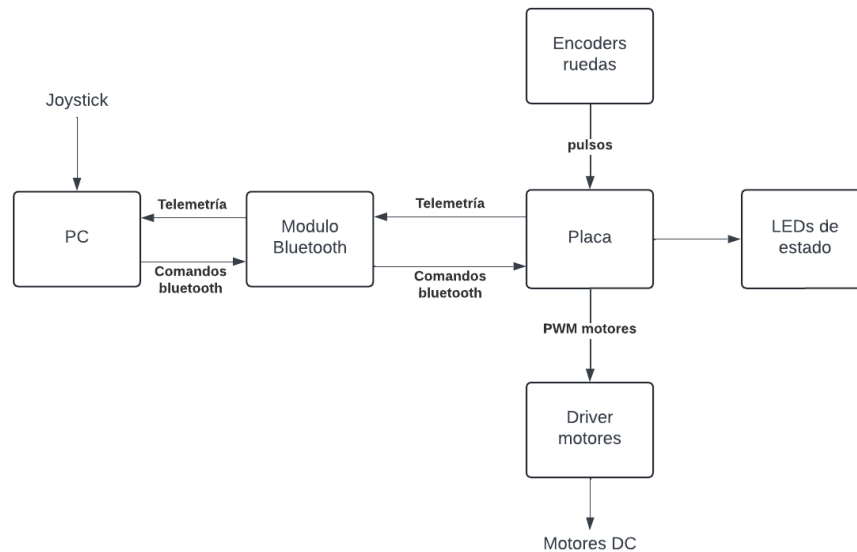


Figure 1: diagrama de bloques

Entradas y salidas

Entradas

- Sensores:
 - Cuatro sensores ópticos de horquilla.
- Interfaz de usuario:
 - Un joystick para control de aceleración, entre otros. El joystick se comunica con la PC, que es la cual manda los mensajes al auto.

- El software de PC también tiene controles en la IU para omitir el joystick.
- Señales de red:
 - Comandos bluetooth que se originan en una PC para controlar el auto.

Salidas

- Motores: dos motores DC. Se controlará la potencia de estos mediante dos señales PWM.
- Habrán varios LEDs sobre el auto que indicarán distintas condiciones, como detección de deslizamiento, control de tracción activado, etc.
- El auto enviará información mediante bluetooth para visualizarla en la PC.

Requisitos de tiempo real

La detección del deslizamiento de las ruedas (y el ajuste de potencia de los motores) requiere de tiempos de procesamiento muy cortos y consistentes. Esta sería la sección crítica. Por otro lado, un apartado no tan importante con respecto a los tiempos sería la recepción de comandos bluetooth. Por último, la telemetría representa una tarea intensiva en términos de procesamiento (ya que se utilizarán funciones de transmisión bloqueantes), pero de muy baja prioridad.

Arquitectura de Software y diseño en FreeRTOS

Tareas (en orden de prioridad, mayor a menor):

- Tarea de control de motores: se encarga de detectar el deslizamiento, y de implementar el control PID para la potencia del motor. También tiene en cuenta el control de velocidad en caso de que este se encuentra activado.
- Tarea de recepción de comandos: se encargar de recibir comandos de control de la PC mediante bluetooth, y procesarlos.
- Tarea de telemetría: se encarga de enviar información mediante bluetooth a la PC para su visualización.

Sincronización y comunicación

- Comunicación entre tareas:
 - La tarea de recepción de comandos, en base a los comandos bluetooth recibidos, le enviará información a la tarea de control de motores utilizando variables compartidas.
 - La tarea de control de motores escribirá continuamente en una estructura compartida, que la tarea de telemetría leerá al momento de enviar un paquete a la PC.
 - No se implementará exclusión mutua para ninguno de estos datos compartidos, puesto a que por como está implementado, no es crítico

para el buen funcionamiento del sistema. Por ejemplo, no pasa nada si la tarea de telemetría manda un paquete que mezcla información vieja con nueva, puesto que es sólo para visualización.

- Manejo de interrupciones (ISRs):
 - Se configurará un timer que generará interrupciones cada cierto tiempo para despertar la tarea de control de motores. El ISR lo único que hace es incrementar un semáforo, para así devolver el control al scheduler lo antes posible.
 - Se generarán interrupciones cuando se reciban datos mediante bluetooth (UART). El ISR correspondiente se encarga de interpretar el stream de bytes como mensajes discretos, y los manda mediante una Queue a la tarea de recepción de comandos.

Arquitectura de Hardware

- Hardware: NUCLEO-H533RE.
- Módulo bluetooth HC-06.
- Driver motores DC L298N.
- Cuatro sensores ópticos FC-03.
- Timers:
 - 1 timer para los cuatro encoders incrementales de las ruedas, con un canal por cada encoder configurado en modo de captura.
 - 1 timer que define la frecuencia en la que se ejecuta la tarea de control de motores.
 - 1 UART para la comunicación bluetooth serial.
- LEDs de distintos colores.

Metodología de validación

- Control de tracción: consiste en poner un objeto a remontar por el auto. Este objeto debe ser lo suficientemente pesado como para causar la pérdida de tracción de las ruedas. Al activar el control de tracción, se puede apreciar tanto con el ojo humano como mediante la telemetría graficada en la PC el control de tracción actuar.
- Control de velocidad: mediante dos botones del joystick se puede incrementar y decrementar la velocidad deseada del auto. Para verificación más estricta, en la visualización de la telemetría se puede comparar la velocidad deseada con la velocidad medida.

Problemas encontrados durante el desarrollo

Primero probé unos encoders incrementales de cuadratura, con unos discos ranurados de 100 PPR (pulsos por revolución). Nunca pude hacer andar la cuadratura puesto que no había forma de posicionar el encoder de manera que se generen ambas señales de pulsos, solo se generaba una a la vez. Entonces decidí usar una sola señal, pero tenía dos nuevos problemas: uno es que de

vez en cuando se perdían pulsos, y el otro es que los anchos de pulsos no eran consistentes, lo que hacía que la medición de RPM fuera ruidosa. Además el ensamblado era extremadamente sensible.

Después me compré otros sensores ópticos más simples (FC-03), junto con unos discos ranurados de 20 PPR. Estos resultaron muchísimo más simples de ensamblar y de hacer andar, pero seguían teniendo el problema de la señal ruidosa. Esto lo compensé mediante una media móvil exponencial, pero aún así limita las posibilidades del proyecto.

Algunas decisiones de diseño

- La tarea de telemetría consiste de un bucle infinito que manda paquete tras paquete de información, sin ningún tipo de espera / delay. Esto no representa ningún problema puesto que esta tarea es la de menor prioridad.
- Con respecto al uso de variables compartidas, es difícil justificarlo sin mirar el código, pero esencialmente el argumento es que no existe ninguna estructura compleja de datos que tenga una “invariante fuerte” a mantener. Por ejemplo, la tarea de recepción de comandos bluetooth modifica la variable de “throttle”, que especifica la aceleración deseado por el usuario. Esta variable es un float, y las escrituras y lecturas a un float son atómicas. Tampoco importa que la tarea de control de motores lea el valor más reciente siempre, se puede permitir demorarse un ciclo sin ningún problema. La mayoría de casos son así. Un caso ligeramente distinto es el de la telemetría: la tarea de control de motores va escribiendo en esta estructura a lo largo de cada iteración, mientras que la tarea de telemetría al terminar de enviar un paquete, inmediatamente arma un nuevo paquete con la información más reciente y lo empieza a transmitir. O sea que puede ser que en algún momento se mande un paquete de telemetría que mezcle información de dos iteraciones distintas del bucle de la tarea de control de motores. Esto no es un problema pues la telemetría es sólo para visualización.
- Justificación de las prioridades de las tareas:
 1. En primer lugar se ubica la tarea de control de motores. Esto se debe a que esta tarea implementa un PID, lo cual para su correcto funcionamiento requiere de intervalos de procesamiento lo más consistentes posibles. Sería indeseable por ejemplo que un ciclo del bucle sea ralentizado por culpa de procesar un mensaje bluetooth. Además, esta tarea es relativamente ligera en términos de procesamiento por lo cual no produce starvation de las otras tareas.
 2. En segundo lugar se encuentra la tarea de recepción de comandos, la cual se encarga de efectuar los comandos que llegan por bluetooth. Entre otras cosas, esta tarea utiliza varios printf para imprimir en la consola los comandos recibidos, por lo cual es una tarea altamente bloqueante (durante los momentos en que recibe comandos). Además, no es imperativo que los comandos posean la menor latencia posible.

Por decir un ejemplo, sería casi imperceptible que el auto demore 50ms (número irrealista) en reaccionar al input del usuario. Por estas razones esta tarea se encuentra en prioridad n°2.

3. En tercer y último lugar se ubica la tarea de telemetría. Esta tarea es 100% bloqueante pues manda paquetes bluetooth constantemente mediante la función bloqueante `HAL_UART_Transmit`. O sea que en principio no puede tener una prioridad más alta que la tarea de recepción de comandos, ya que si no causaría starvation de esta tarea. Además tampoco existe ninguna necesidad de que los paquetes de telemetría lleguen en intervalos consistentes o con baja latencia, por lo que no tendría sentido que la tarea de recepción de comandos se demore por culpa de la telemetría. Luego, es lógico poner esta tarea con prioridad n°3.